

Name: Priya J. Shah

ID: 202512100

- 142246

Data Structures IT623 (Autumn 2025 - 2026)
First In-semester Examination
Dhirubhai Ambani University, Gandhinagar, Gujarat, India

Duration: 90 Minutes

Date: August 20, 2025

Total Marks: 30

Instructions: Write your name and ID on all pages, failing so your answer sheet will not be evaluated. All questions are compulsory. Answers must have proper justifications to get full credit. Write your answer immediately next to the question on the same page. Additional sheets will not be evaluated.

1. [6 marks] Give the tightest possible Big-O estimate for the following expression:

$$(n^2 + 5n \log n)(\log n + 10) + (n^2 + 2n + 3)(n + 8) \cdot \frac{\log n}{n} (\log^2 n + 9) + \log(n!) + (10n^2 + 5n + 2) \log^2 n$$

Show all steps and reasoning.

$$\begin{aligned} &= n^2 \cdot \log n + 10n^2 + 5n \cdot \log^2 n + 50n \log n + n^3 + 8n^2 + 2n^2 + 16n + 3n + 24 \cdot \frac{\log n}{n} (\log^2 n) + \frac{\log n}{n} (9) + \log(n!) + 10n^2 \log^2 n + 5n \log^2 n + 2 \log^2 n \\ &= n^2 + [n^3 + 10n^2 + 19n + 24 \cdot \frac{\log n}{n} (\log^2 n) + \frac{\log n}{n} (9)] + \log(n!) + n^2 \\ &= n^2 + [n^3 + \frac{\log n}{n} (\log^2 n)] + \log(n!) + n^2 \\ &= n^2 + n^3 + \frac{\log n}{n} (\log^2 n) + \log(n!) + n^2 \\ &= 2n^2 + n^3 + \frac{\log n}{n} (\log^2 n) + \log(n!) \end{aligned}$$

Tightest possible Big-O — n^3

$O(n^3)$

(2)

→ In step 1, 3, 5 we eliminated the lowest degrees of the equation while moving ahead with highest degree to get the most accurate and tightest possible Big-O.

2. [6 marks] Find both best and worst case running time $T(n)$ for the following pseudocode and express using asymptotic notation (i.e., after solving the recurrence). Assume A contains n elements and "AuxFunc" procedure return an integer q between p and r and takes $O(n)$ time. Provide a detailed and clear justification.

TESTPROC(A, p, r, k)	_____	$T(n)$
1 if $n \leq 1$	_____	1
2 return k	_____	n
3 for $i = p$ to r	_____	$\log n$
4 for $j = p$ to $r/2$	_____	$n * \log n$
5 print($A[i]$)	_____	$O(n)$
6 $q = \text{AuxFunc}(A, p, r)$	_____	$T(n-1)$
7 TESTPROC($A, p, q-1, k$)	_____	$T(n-1)$
8 TESTPROC($A, q+1, r, n-k$)	_____	$T(n-1)$

Time Complexity:

$$\begin{aligned}
 T(n) &= T(n-1) + T(n-1) + n + n * \log n + \log n + n + 1 \\
 &= 2T(n-1) + n * \log n + \log n + 2n + 1 \\
 &= 2T(n-1) + \log n (n+1) + 2n + 1
 \end{aligned}$$

Best case Time Complexity: ~~$O(n)$~~ $O(1)$

- The best case is when the array is empty, so an being called it will immediately return the value before proceeding further
- In this pseudocode, if the array is empty having atmost 2 elements all the loop statements and recursive function calls will be eliminated.

Worst case Time Complexity:

- The worst case scenario the loop runs infinitely if the q generated by AuxFunction is never resulting the array with only 2 elements.
- In that situation this algorithm might fail and runs endlessly resulting in stack overflow

3. [6 marks] Consider the recurrence

$$T(n) = 3T\left(\frac{n}{4}\right) + n \log n$$

with $T(1) = 1$. Solve this recurrence and find the tight asymptotic bound for $T(n)$ using master method.Master Method general form: $aT\left(\frac{n}{b}\right) + \text{mag}n f(n)$

→ As $3T\left(\frac{n}{4}\right) + n \log n$ fits in the general form of master method, it is applicable here.

$$a = 3, b = 4, f(n) = n \log n$$

$$f(n) + v(n)$$

where $v(n) = h(n)$.

$$h(n) = \frac{f(n)}{\log_b a}$$

$$h(n) = \frac{n \log n}{\log_4 3}$$

2.5

4. [6 marks] A 2D array $A[m][n]$ is stored in column-major order with base address M , and each element occupies w bytes.

- (a) (3 marks) Derive the formula to compute the memory address of element $A[i][j]$ for $0 \leq i < m$, $0 \leq j < n$.
- (b) (3 marks) Using your formula, compute the address of $A[20][10]$ when $A[50][40]$ is stored with base address $M = 115$ and $w = 3$ bytes. Show all steps.

a) Derivation of formula to compute memory address of element $A[i][j]$

$$\text{Address of } A[i][j] = M + (j * m + i) * w$$

$\swarrow$ base address $\downarrow$ No. of elements in each column $\downarrow$ bytes occupied
 $\swarrow$ column number $\downarrow$ row number

→ Adding the base address

a_{00}	Column 0
a_{10}	
a_{20}	
a_{30}	
a_{01}	Column 1
a_{11}	
a_{21}	
a_{31}	
a_{02}	Column 2
a_{12}	
a_{22}	
a_{32}	
$\vdots$	Column ...
a_{m2}	

$$\begin{aligned}
 \text{b) } A[20][10] &= 115 + (10 * 50 + 20) * 3 \\
 &= 115 + (520) * 3 \\
 &= 115 + 1560 \\
 &= \underline{1675}
 \end{aligned}$$

$$\begin{aligned}
 A[50][40] &= 115 + (40 * 50 + 50) * 3 \\
 &= 115 + (2050) * 3 \\
 &= 115 + 6150 \\
 &= \underline{6265}
 \end{aligned}$$

6

5. [6 marks] Given a sorted array $A[1 \dots n]$ of distinct integers, design an efficient algorithm to determine whether there exist two elements in A whose sum is equal to a given target value T . Provide pseudocode, explain your approach, and analyze its running time.

Pseudocode :

```

bool check(A, T) *            $T(n)$ 
{
    for(i=0; i < n-1; i++)     $(n-1)$ 
        for(j=i+1; j < n; j++)  $(n-1)(n-i)$ 
            if (A[i] + A[j] == T)
                return true     $1$ 
    return false               $1$ 
}

```

Approach :

- 2 nested for loops will be running, checking the sum of one element with all other elements
- ~~loop is~~ Outer loop will stop at $n-1$ to not recheck the last element as it appears in the round of other elements
- Inner loop will ~~start~~ always one step ahead from outer to avoid testing same elements

Time Complexity:

$$\rightarrow T(n) = (n-1) + (n-1)(n-i) + 1 + 1$$

$$T(n) = (n-1) + n^2 - ni - n + i + 2$$

$$T(n) = (n-1) + n^2 - ni - n + i$$

$O(n^2)$ - Big Oh Asymptotic Notation

Runs for maximum n^2 if the loop doesn't find target T till the ~~last 2~~ elements or they don't exist in the array.